

Медицинская информационная система ведения терапии варфарином

Распечатка фрагментов исходного кода
(титульный лист оформляется патентоведом)

api/src/Ibs/Context/Communication/Service/ChannelInterface.php

<?php

```
declare(strict_types=1);
```

```
namespace Ibs\Context\Communication\Service;
```

```
use Ibs\Context\Communication\Model\NotificationMessage;
```

```
use Ibs\Context\Communication\Model\Recipient;
```

```
use Ibs\Context\Communication\Model\SendResult;
```

```
use Symfony\Component\DependencyInjection\Attribute\AutoconfigureTag;
```

```
/**
```

```
 * Контракт канала связи. Реализации регистрируются как сервисы  
 * с тегом
```

```
 * 'communication.channel' (автоматически, благодаря AutoconfigureTag на этом
```

```
 * интерфейсе) и подхватываются NotificationService через ChannelRegistry –
```

```
 * добавление нового канала не требует изменений в вызывающем ко  
 * де.
```

```
 */
```

```
#[AutoconfigureTag('communication.channel')]
```

```
interface ChannelInterface
```

```
{
```

```
    public function send(Recipient $recipient, NotificationMessage $message): SendResult;
```

```
    public function isAvailable(): bool;
```

```
    /**
```

```
     * @return string например 'sms', 'push', 'email', 'max'
```

```
     */
```

```
    public function getChannelType(): string;
```

```
}
```

api/src/Ibs/Context/Communication/Service/ChannelRegistry.php

<?php

```
declare(strict_types=1);
```

```
namespace Ibs\Context\Communication\Service;
```

```
use Symfony\Component\DependencyInjection\Attribute\AutowireIterator;
```

```
/**
```

```
 * Реестр всех зарегистрированных каналов связи, проиндексированных по типу
```

```
 * канала. Собирает все сервисы с тегом 'communication.channel'
```

```

        – новый канал
    * достаточно зарегистрировать как сервис, реализующий ChannelIn
      terface.
    */
final class ChannelRegistry
{
    /** @var array<string, ChannelInterface> */
    private readonly array $channelsByType;

    /**
     * @param iterable<ChannelInterface> $channels
     */
    public function __construct(
        #[AutowireIterator('communication.channel')]
        iterable $channels = [],
    ) {
        $channelsByType = [];
        foreach ($channels as $channel) {
            $channelsByType[$channel->getChannelType()] = $chann
el;
        }
        $this->channelsByType = $channelsByType;
    }

    public function get(string $channelType): ?ChannelInterface
    {
        return $this->channelsByType[$channelType] ?? null;
    }

    public function has(string $channelType): bool
    {
        return isset($this->channelsByType[$channelType]);
    }
}
api/src/Ibs/Context/Communication/Service/Exception/ChannelNotFoundExcept
<?php

declare(strict_types=1);

namespace Ibs\Context\Communication\Service\Exception;

/**
 * Выбрасывается при запросе канала, для которого не зарегистрир
   ован сервис с тегом
 * 'communication.channel'.
 */
class ChannelNotFoundException extends \RuntimeException
{
    public function __construct(string $channelType)
    {

```

```

        parent::__construct(sprintf('Notification channel "%s"
        is not registered.', $channelType));
    }
}
api/src/Ibs/Context/Communication/Service/Exception/NoChannelsConfiguredE
<?php

declare(strict_types=1);

namespace Ibs\Context\Communication\Service\Exception;

/**
 * Выбрасывается, если NotificationService::send() вызван без ук
 * азания каналов доставки.
 */
class NoChannelsConfiguredException extends \RuntimeException
{
    public function __construct()
    {
        parent::__construct('No notification channels were confi
        gured for this message.');
```

```

    }
}
api/src/Ibs/Context/Communication/Service/Exception/NotificationDeliveryE
<?php

declare(strict_types=1);
```

```

namespace Ibs\Context\Communication\Service\Exception;

/**
 * Выбрасывается TemplateResolver'ом, если для указанного кода и
   канала не найден NotificationTemplate.
 */
class TemplateNotFoundException extends \RuntimeException
{
    public function __construct(string $templateCode, string $channelType)
    {
        parent::__construct(sprintf('Notification template "%s"
        for channel "%s" was not found.', $templateCode, $channelType));
    }
}
api/src/Ibs/Context/Communication/Service/Message/SendNotificationEnvelope.php
<?php

declare(strict_types=1);

namespace Ibs\Context\Communication\Service\Message;

use Ibs\Context\Communication\Model\NotificationMessage;
use Ibs\Context\Communication\Model\Priority;
use Ibs\Context\Communication\Model\Recipient;

/**
 * Сообщение Messenger для асинхронной (ROUTINE) доставки уведомления.
 * Обрабатывается NotificationService::handleEnvelope().
 */
final class SendNotificationEnvelope
{
    /**
     * @param string[] $channels
     */
    public function __construct(
        public readonly Recipient $recipient,
        public readonly NotificationMessage $message,
        public readonly array $channels,
        public readonly Priority $priority,
    ) {
    }
}
api/src/Ibs/Context/Communication/Service/NotificationService.php
<?php

declare(strict_types=1);

```

```

namespace Ibs\Context\Communication\Service;

use Ibs\Context\Communication\Entity\NotificationLog;
use Ibs\Context\Communication\Model\NotificationMessage;
use Ibs\Context\Communication\Model\Priority;
use Ibs\Context\Communication\Model\Recipient;
use Ibs\Context\Communication\Model\SendResult;
use Ibs\Context\Communication\Repository\NotificationLogRepository;
use Ibs\Context\Communication\Service\Exception\ChannelNotFoundException;
use Ibs\Context\Communication\Service\Exception\NoChannelsConfiguredException;
use Ibs\Context\Communication\Service\Exception\NotificationDeliveryException;
use Ibs\Context\Communication\Service\Message\SendNotificationEnvelope;
use Symfony\Component\Messenger\Attribute\AsMessageHandler;
use Symfony\Component\Messenger\MessageBusInterface;

/**
 * Единая точка отправки уведомлений через любые каналы связи (SMS, Push, Email, MAX).
 * Скрывает от клиентов детали конкретных каналов, применяет приоритет доставки
 * и логирует каждую попытку отправки в NotificationLog.
 */
final class NotificationService
{
    public function __construct(
        private readonly ChannelRegistry $channels,
        private readonly TemplateResolver $templateResolver,
        private readonly NotificationLogRepository $logRepository,
        private readonly ?MessageBusInterface $messageBus = null
    ) {
    }

    /**
     * @param string[] $channels типы каналов, например ['sms', 'email']
     */
    public function send(
        Recipient $recipient,
        NotificationMessage $message,
        array $channels = [],
        Priority $priority = Priority::ROUTINE,
    ): void {
        if ([] === $channels) {

```

```

        $exception = new NoChannelsConfiguredException();
        $this->log($recipient, null, $priority, $message->template, 'failed', $exception->getMessage());
        throw $exception;
    }

    if (Priority::IMMEDIATE === $priority) {
        // Критические алерты: отправляем синхронно, ошибка
        // пробрасывается вызывающему коду.
        $this->deliver($recipient, $message, $channels, $priority, rethrow: true);
        return;
    }

    if (null !== $this->messageBus) {
        $this->messageBus->dispatch(new SendNotificationEnvelope($recipient, $message, $channels, $priority));
        return;
    }

    // Очередь недоступна – fallback на синхронную отправку,
    // ошибки только логируются.
    $this->deliver($recipient, $message, $channels, $priority, rethrow: false);
}

/**
 * Обработчик асинхронной доставки ROUTINE-уведомлений через
 * Messenger.
 */
#[AsMessageHandler]
public function handleEnvelope(SendNotificationEnvelope $envelope): void
{
    $this->deliver($envelope->recipient, $envelope->message,
        $envelope->channels, $envelope->priority, rethrow: false);
}

/**
 * @return NotificationLog[]
 */
public function getHistoryForPatient(int $patientId): array
{
    return $this->logRepository->findByPatient($patientId);
}

/**
 * @return NotificationLog[]
 */
public function getHistoryForTreatment(int $treatmentId): array

```

```

ray
{
    return $this->logRepository->findByTreatment($treatmentI
d);
}

/**
 * @param string[] $channelTypes
 */
private function deliver(
    Recipient $recipient,
    NotificationMessage $message,
    array $channelTypes,
    Priority $priority,
    bool $rethrow,
): void {
    try {
        foreach ($channelTypes as $channelType) {
            $channel = $this->channels->get($channelType);

            if (null === $channel) {
                $this->log($recipient, $channelType, $priori
ty, $message->template, 'failed', \sprintf('Channel "%s" is
not registered.', $channelType), flush: false);
                if ($rethrow) {
                    throw new ChannelNotFoundException($chan
nelType);
                }
                continue;
            }

            if (!$channel->isAvailable()) {
                // Пропускаем resolve()/send() заранее – не
ждём таймаута от заведомо недоступного канала.
                $this->log($recipient, $channelType, $priori
ty, $message->template, 'failed', \sprintf('Channel "%s" is
not available.', $channelType), flush: false);
                if ($rethrow) {
                    throw new NotificationDeliveryException(
$channelType, 'Channel is not available.');
```

```

        $result = SendResult::failure($exception->getMessage());
    }

    $this->log($recipient, $channelType, $priority,
$message->template, $result->success ? 'sent' : 'failed', $result->errorMessage, flush: false);

        if (!$result->success && $rethrow) {
            throw new NotificationDeliveryException($channelType, $result->errorMessage ?? 'Unknown error');
        }
    } finally {
        // Один flush на все каналы вместо flush на каждую запись – срабатывает и при throw выше.
        $this->logRepository->flush();
    }
}

private function log(
    Recipient $recipient,
    ?string $channelType,
    Priority $priority,
    ?string $templateCode,
    string $status,
    ?string $errorMessage,
    bool $flush = true,
): void {
    $log = (new NotificationLog())
        ->setPatientId($recipient->patientId)
        ->setTreatmentId($recipient->treatmentId)
        ->setChannelType($channelType)
        ->setRecipientAddress(null !== $channelType ? $recipient->addressFor($channelType) : null)
        ->setPriority($priority->value)
        ->setTemplateCode($templateCode)
        ->setStatus($status)
        ->setErrorMessage($errorMessage)
        ->setCreatedAt(new \DateTimeImmutable());

    $this->logRepository->save($log, $flush);
}
}

```

api/src/Ibs/Context/Communication/Service/TemplateResolver.php
<?php

```

declare(strict_types=1);

namespace Ibs\Context\Communication\Service;

```

```

use Ibs\Context\Communication\Model\NotificationMessage;
use Ibs\Context\Communication\Repository\NotificationTemplateRepository;
use Ibs\Context\Communication\Service\Exception\TemplateNotFoundException;

/**
 * Разрешает шаблон уведомления по коду (NotificationTemplate) и
 * подставляет
 * плейсхолдеры вида %key% значениями из NotificationMessage::$data. Если код
 * шаблона не указан, подстановка выполняется прямо в body/subject сообщения.
 */
final class TemplateResolver
{
    public function __construct(
        private readonly NotificationTemplateRepository $templates,
    ) {
    }

    public function resolve(NotificationMessage $message, string $channelType): NotificationMessage
    {
        $bodyTemplate = $message->body;
        $subjectTemplate = $message->subject;

        if (null !== $message->template) {
            $template = $this->templates->findOneByCodeAndChannel($message->template, $channelType);
            if (null === $template) {
                throw new TemplateNotFoundException($message->template, $channelType);
            }

            $bodyTemplate = $template->getBodyTemplate() ?? '';
            $subjectTemplate = $template->getSubjectTemplate() ? $subjectTemplate;
        }

        return new NotificationMessage(
            body: $this->substitutePlaceholders($bodyTemplate, $message->data),
            subject: null !== $subjectTemplate ? $this->substitutePlaceholders($subjectTemplate, $message->data) : null,
            template: $message->template,
            data: $message->data,
            attachments: $message->attachments,
        );
    }
}

```

```

        );
    }

    /**
     * @param array<string, scalar|null> $data
     */
    private function substitutePlaceholders(string $text, array
    $data): string
    {
        $result = preg_replace_callback(
            '/%([a-zA-Z0-9_]+)%/',
            static function (array $matches) use ($data): string
            {
                // Оставляем плейсхолдер как есть, если данные д
                ля него не переданы.
                return \array_key_exists($matches[1], $data) ? (
                string) $data[$matches[1]] : $matches[0];
            },
            $text,
        );

        // preg_replace_callback() возвращает null при ошибке PC
        RE – не подставляем ничего, а не падаем с TypeError.
        return $result ?? $text;
    }
}
api/src/Ibs/Context/TreatmentTherapy/State/AppointmentSaveProcessor.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\State\ProcessorInterface;
use Ibs\Context\TreatmentTherapy\Entity\Appointment;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\SecurityBundle\Security;
use Symfony\Component\DependencyInjection\Attribute\Autowire;
use Symfony\Component\HttpKernel\Exception\UnprocessableEntityHt
tpException;

class AppointmentSaveProcessor implements ProcessorInterface
{
    private EntityManagerInterface $entityManager;

    public function __construct(
        #[Autowire('@api_platform.doctrine.orm.state.persist_pro
cessor')]
        private ProcessorInterface $persistProcessor,

```

```

        private Security $security,
        EntityManagerInterface $entityManager
    ) {
        $this->entityManager = $entityManager;
    }

    public function process(mixed $data, Operation $operation, array $uriVariables = [], array $context = []): Appointment
    {
        /** @var Appointment $data */
        $treatment = $data->getTreatment();

        // Проверка активности лечения
        if ($treatment === null || $treatment->getRealEndDt() !=
= null) {
            throw new UnprocessableEntityHttpException('Лечение
не активно. Сохранение назначения невозможно.');
```

```
        }

        // Автоматическое заполнение дат
        $now = new \DateTime();
        if ($data->getCreationDt() === null) {
            $data->setCreationDt($now);
        }
        $data->setModDt($now);

        // Проверка 50% изменения дозы
        $lastAppointment = $this->entityManager->getRepository(A
ppointment::class)
            ->findOneBy(
                ['treatment' => $treatment],
                ['appointmentDt' => 'DESC']
            );
        if ($lastAppointment) {
            $lastDose = $lastAppointment->getDoze();
            $newDose = $data->getDoze();
            if ($lastDose > 0) {
                $change = abs($newDose - $lastDose) / $lastDose;
                if ($change > 0.5) {
                    error_log(sprintf(
                        'Предупреждение: изменение дозы более 50
%% для лечения %d, старая доза %.2f, новая доза %.2f',
                        $treatment->getId(),
                        $lastDose,
                        $newDose
                    ));
                }
            }
        }
    }
}

```

```

        // Автоматическое заполнение врача
        $user = $this->security->getUser();
        if ($user) {
            $data->setDoctorName($user->getUserIdentifier());
        }

        return $this->persistProcessor->process($data, $operation, $uriVariables, $context);
    }
}
api/src/Ibs/Context/TreatmentTherapy/State/Mkb10PopularProvider.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\State\ProviderInterface;
use Ibs\Context\TreatmentTherapy\Repository\Mkb10Repository;

class Mkb10PopularProvider implements ProviderInterface
{
    public function __construct(private Mkb10Repository $repository)
    {
    }

    public function provide(Operation $operation, array $uriVariables = [], array $context = []): array
    {
        return $this->repository->findPopularActiveDiagnoses(10);
    }
}
api/src/Ibs/Context/TreatmentTherapy/State/Mkb10SearchProvider.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\State\ProviderInterface;
use Ibs\Context\TreatmentTherapy\Repository\Mkb10Repository;
use Symfony\Component\HttpFoundation\RequestStack;

class Mkb10SearchProvider implements ProviderInterface
{
    public function __construct(

```

```

        private Mkb10Repository $repository,
        private RequestStack $requestStack
    ) {}

    public function provide(Operation $operation, array $uriVariables = [], array $context = []): array
    {
        $request = $this->requestStack->getCurrentRequest();
        $query = $request->query->get('q', '');
        if (strlen($query) < 2) {
            return [];
        }

        return $this->repository->searchByCodeOrName($query);
    }
}
api/src/Ibs/Context/TreatmentTherapy/State/PatientStatusProcessor.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\State\ProcessorInterface;
use Ibs\Context\TreatmentTherapy\Dto\PatientStatus\PatientStatusRequest;
use Ibs\Context\TreatmentTherapy\Dto\PatientStatus\PatientStatusResponse;
use Ibs\Context\TreatmentTherapy\Repository\TreatmentRepository;

final class PatientStatusProcessor implements ProcessorInterface
{
    private const ACTIVE_PATIENT_STATUS = 'активный';
    private const CANCELLED_PATIENT_STATUS = 'неактивный';

    public function __construct(
        private TreatmentRepository $treatmentRepository
    ) {}

    public function process(mixed $data, Operation $operation, array $uriVariables = [], array $context = [])
    {
        /** @var PatientStatusRequest $data */
        $patientIds = $data->ids;

        $activeIds = $this->treatmentRepository->getActivePatientIds($patientIds);

        $result = [];

```

```

        foreach ($patientIds as $id) {
            $status = in_array($id, $activeIds) ? self::ACTIVE_P
            ATIENT_STATUS : self::CANCELLED_PATIENT_STATUS;
            $result[] = new PatientStatusResponse($id, $status);
        }

        return $result;
    }
}
api/src/Ibs/Context/TreatmentTherapy/State/TestHistoryLatestProvider.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\State;

use ApiPlatform\Metadata\Operation;
use ApiPlatform\State\ProviderInterface;
use Ibs\Context\TreatmentTherapy\Repository\TestHistoryRepository;
use Symfony\Component\HttpFoundation\RequestStack;

class TestHistoryLatestProvider implements ProviderInterface
{
    public function __construct(
        private TestHistoryRepository $repository,
        private RequestStack $requestStack
    ) {}

    public function provide(Operation $operation, array $uriVariables = [], array $context = []): array
    {
        $request = $this->requestStack->getCurrentRequest();
        $treatmentIds = $request->query->all('treatment');
        if (empty($treatmentIds)) {
            return [];
        }
        $treatmentIds = array_map('intval', $treatmentIds);
        return $this->repository->findLatestByTreatmentIds($treatmentIds);
    }
}
api/src/Ibs/Context/TreatmentTherapy/Resource/PatientStatusResource.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Resource;

use ApiPlatform\Metadata\Post;

```

```

use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\OpenApi\Model\Operation as OpenApiOperation;
use ApiPlatform\OpenApi\Model\RequestBody;
use ApiPlatform\OpenApi\Model\Response;
use Ibs\Context\TreatmentTherapy\Dto\PatientStatus\PatientStatus
    Response;
use Ibs\Context\TreatmentTherapy\Dto\PatientStatus\PatientStatus
    Request;
use Ibs\Context\TreatmentTherapy\State\PatientStatusProcessor;

#[ApiResource(
    operations: [
        new Post(
            uriTemplate: '/patients/status',
            status: 200,
            openapi: new OpenApiOperation(
                tags: ['Пациенты'],
                responses: [
                    '200' => new Response(
                        description: 'Успешный ответ',
                        content: new \ArrayObject([
                            'application/json' => [
                                'schema' => [
                                    'type' => 'array',
                                    'items' => [
                                        'type' => 'object',
                                        'properties' => [
                                            'id' => ['type' => '
integer'],
                                            'status' => ['type'
=> 'string', 'enum' => ['активный', 'неактивный']]
                                        ]
                                    ]
                                ]
                            ]
                        ])
                    ),
                    '400' => new Response(description: 'Неверный
запрос (например, не передан ids)'),
                    '422' => new Response(description: 'Ошибка в
алидации (некорректные ID)'),
                ],
                summary: 'Получить статусы пациентов',
                description: 'Возвращает статусы (активный/неакт
ивный) для переданных ID пациентов. Не изменяет состояние си
стемы.',
                requestBody: new RequestBody(
                    content: new \ArrayObject([
                        'application/json' => [
                            'schema' => [

```

```

        'type' => 'object',
        'properties' => [
            'ids' => [
                'type' => 'array',
                'items' => ['type' => 'integer'],
                'example' => [1, 2, 3],
                'description' => 'Массив ID пациентов'
            ],
            'required' => ['ids']
        ]
    ],
    input: PatientStatusRequest::class,
    output: PatientStatusResponse::class,
    processor: PatientStatusProcessor::class
)
]
)]
class PatientStatusResource
{
}
api/src/Ibs/Context/Communication/Repository/NotificationLogRepository.php
<?php

declare(strict_types=1);

namespace Ibs\Context\Communication\Repository;

use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepository;
use Doctrine\Persistence\ManagerRegistry;
use Ibs\Context\Communication\Entity\NotificationLog;

/**
 * @extends ServiceEntityRepository<NotificationLog>
 */
class NotificationLogRepository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, NotificationLog::class);
    }

    public function save(NotificationLog $log, bool $flush = true): void

```

```

    {
        $entityManager = $this->getEntityManager();
        $entityManager->persist($log);
        if ($flush) {
            $entityManager->flush();
        }
    }

    public function flush(): void
    {
        $this->getEntityManager()->flush();
    }

    /**
     * @return NotificationLog[]
     */
    public function findByPatient(int $patientId): array
    {
        return $this->findBy(['patientId' => $patientId], ['crea
tedAt' => 'DESC']);
    }

    /**
     * @return NotificationLog[]
     */
    public function findByTreatment(int $treatmentId): array
    {
        return $this->findBy(['treatmentId' => $treatmentId], ['
createdAt' => 'DESC']);
    }
}
api/src/Ibs/Context/Communication/Repository/NotificationTemplateRepository.php
<?php

declare(strict_types=1);

namespace Ibs\Context\Communication\Repository;

use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepos
itory;
use Doctrine\Persistence\ManagerRegistry;
use Ibs\Context\Communication\Entity\NotificationTemplate;

/**
 * @extends ServiceEntityRepository<NotificationTemplate>
 */
class NotificationTemplateRepository extends ServiceEntityReposi
tory
{
    public function __construct(ManagerRegistry $registry)

```

```

    {
        parent::__construct($registry, NotificationTemplate::cla
ss);
    }

    public function findOneByCodeAndChannel(string $code, string
        $channel): ?NotificationTemplate
    {
        return $this->findOneBy(['code' => $code, 'channel' => $
channel]);
    }
}
api/src/Ibs/Context/TreatmentTherapy/Repository/Mkb10Repository.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Repository;

use Ibs\Context\TreatmentTherapy\Entity\Mkb10;
use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepos
    itory;
use Doctrine\Persistence\ManagerRegistry;

class Mkb10Repository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Mkb10::class);
    }

    public function findPopularActiveDiagnoses(int $limit = 10):
        array
    {
        $conn = $this->getEntityManager()->getConnection();
        $sql = '
            SELECT
                m.id,
                m.mkb_code,
                m.mkb_name,
                COUNT(t.id) as cnt
            FROM mkb10 m
                JOIN treatments t ON t.mkb10_id = m.id
            WHERE t.real_end_dt IS NULL
            GROUP BY m.id
            ORDER BY cnt DESC
            LIMIT :limit
        ';
        $stmt = $conn->executeQuery($sql, ['limit' => $limit]);
        $rows = $stmt->fetchAllAssociative();
    }
}

```

```

        $result = [];
        foreach ($rows as $row) {
            $mkb10 = $this->find($row['id']);
            if ($mkb10) {
                $result[] = $mkb10;
            }
        }
        return $result;
    }
}

public function searchByCodeOrName(string $query, int $limit
= 20): array
{
    $conn = $this->getEntityManager()->getConnection();
    $sql = '
        SELECT *
        FROM mkb10
        WHERE mkb_code ILIKE :query
        OR mkb_name ILIKE :query
        ORDER BY
            CASE
                WHEN mkb_code = :query_exact THEN 1
                WHEN mkb_code ILIKE :query_start THEN 2
                ELSE 3
            END,
            mkb_code
        LIMIT :limit
    ';
    $stmt = $conn->executeQuery($sql, [
        'query' => "%$query%",
        'query_exact' => $query,
        'query_start' => "$query%",
        'limit' => $limit,
    ]);
    return $stmt->fetchAllAssociative();
}
}

```

api/src/Ibs/Context/TreatmentTherapy/Repository/TestHistoryRepository.php

```

<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Repository;

use Ibs\Context\TreatmentTherapy\Entity\TestHistory;
use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepos
    itory;
use Doctrine\Persistence\ManagerRegistry;

```

```

class TestHistoryRepository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, TestHistory::class);
    }

    public function findLatestByTreatmentIds(array $treatmentIds
): array
    {
        if (empty($treatmentIds)) {
            return [];
        }

        $conn = $this->getEntityManager()->getConnection();
        $sql = '
            SELECT DISTINCT ON (treatment_id) id
            FROM test_history
            WHERE treatment_id IN (:treatmentIds)
            ORDER BY treatment_id, creation_dt DESC, id DESC
        ';
        $stmt = $conn->executeQuery($sql, ['treatmentIds' => $tr
eatmentIds], ['treatmentIds' => \Doctrine\DBAL\ArrayParamete
rType::INTEGER]);
        $ids = $stmt->fetchFirstColumn();

        if (empty($ids)) {
            return [];
        }

        return $this->createQueryBuilder('th')
            ->where('th.id IN (:ids)')
            ->setParameter('ids', $ids)
            ->getQuery()
            ->getResult();
    }
}

```

api/src/Ibs/Context/TreatmentTherapy/Repository/TreatmentRepository.php
<?php

```

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Repository;

use Ibs\Context\TreatmentTherapy\Entity\Treatment;
use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepos
itory;
use Doctrine\Persistence\ManagerRegistry;

/**

```

```

* @extends ServiceEntityRepository<Treatment>
*/
class TreatmentRepository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Treatment::class);
    }

    /**
     * Возвращает ID пациентов, у которых есть хотя бы одно акти
     * вное лечение (realEndDt IS NULL)
     *
     * @param int[] $patientIds
     * @return int[]
     */
    public function getActivePatientIds(array $patientIds): arra
    y
    {
        if (empty($patientIds)) {
            return [];
        }

        $qb = $this->createQueryBuilder('t')
            ->select('IDENTITY(t.patient) as patient_id')
            ->where('t.patient IN (:ids)')
            ->andWhere('t.realEndDt IS NULL')
            ->setParameter('ids', $patientIds);

        $result = $qb->getQuery()->getScalarResult();

        return array_column($result, 'patient_id');
    }
}
api/src/Ibs/Context/TreatmentTherapy/Filter/ActiveTreatmentFilter.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Filter;

use ApiPlatform\Doctrine\Orm\Filter\AbstractFilter;
use ApiPlatform\Doctrine\Orm\Util\QueryNameGeneratorInterface;
use ApiPlatform\Metadata\Operation;
use Doctrine\ORM\QueryBuilder;
use Doctrine\Persistence\ManagerRegistry;
use Psr\Log\LoggerInterface;
use Symfony\Component\Serializer\NameConverter\NameConverterInte
    rface;

```

```

final class ActiveTreatmentFilter extends AbstractFilter
{
    public function __construct(
        ManagerRegistry $managerRegistry,
        ?LoggerInterface $logger = null,
        ?array $properties = null,
        ?NameConverterInterface $nameConverter = null
    ) {
        parent::__construct($managerRegistry, $logger, $properties, $nameConverter);
    }

    protected function filterProperty(
        string $property,
        $value,
        QueryBuilder $queryBuilder,
        QueryNameGeneratorInterface $queryNameGenerator,
        string $resourceClass,
        ?Operation $operation = null,
        array $context = []
    ): void {
        if ($property !== 'active') {
            return;
        }

        // Приводим значение к boolean (можно передать true, 1, 'true', '1')
        $active = filter_var($value, FILTER_VALIDATE_BOOLEAN);
        if ($active) {
            $alias = $queryBuilder->getRootAliases()[0];
            $queryBuilder->andWhere("$alias.realEndDt IS NULL");
        }
    }

    public function getDescription(string $resourceClass): array
    {
        return [
            'active' => [
                'property' => 'active',
                'type' => 'bool',
                'required' => false,
                'description' => 'Фильтр активных лечений (realEndDt IS NULL)',
            ],
        ];
    }
}

```

api/src/Ibs/Context/TreatmentTherapy/Filter/PatientDiagnosisFilter.php
<?php

```

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Filter;

use ApiPlatform\Doctrine\Orm\Filter\AbstractFilter;
use ApiPlatform\Doctrine\Orm\Util\QueryNameGeneratorInterface;
use ApiPlatform\Metadata\Operation;
use Ibs\Context\TreatmentTherapy\Entity\Treatment;
use Doctrine\ORM\QueryBuilder;
use Doctrine\Persistence\ManagerRegistry;
use Psr\Log\LoggerInterface;
use Symfony\Component\Serializer\NameConverter\NameConverterInterface;

final class PatientDiagnosisFilter extends AbstractFilter
{
    public const DIAGNOSIS_FILTER_NAME = 'diagnosisCode';

    public function __construct(
        ManagerRegistry $managerRegistry,
        ?LoggerInterface $logger = null,
        ?array $properties = null,
        ?NameConverterInterface $nameConverter = null
    ) {
        parent::__construct($managerRegistry, $logger, $properties, $nameConverter);
    }

    protected function filterProperty(
        string $property,
        $value,
        QueryBuilder $queryBuilder,
        QueryNameGeneratorInterface $queryNameGenerator,
        string $resourceClass,
        ?Operation $operation = null,
        array $context = []
    ): void {
        if ($property !== self::DIAGNOSIS_FILTER_NAME) {
            return;
        }

        $codes = is_array($value) ? $value : [$value];
        if (empty($codes)) {
            return;
        }

        $parameterName = $queryNameGenerator->generateParameterName('diagnosisCodes');
        $entityManager = $this->managerRegistry->getManager();
    }

```

```

        $subQuery = $entityManager->createQueryBuilder()
            ->select('1')
            ->from(Treatment::class, 't_diag')
            ->innerJoin('t_diag.mkb10', 'm')
            ->where('t_diag.patient = o.id')
            ->andWhere('t_diag.begDt = (
                SELECT MAX(t2_diag.begDt)
                FROM Ibs\Context\TreatmentTherapy\Entity\Treatment
nt t2_diag
                WHERE t2_diag.patient = o.id
            )')
            ->andWhere('t_diag.realEndDt IS NULL')
            ->andWhere('m.mkbCode IN (: ' . $parameterName . ')')
        ;

        $queryBuilder->andWhere($queryBuilder->expr()->exists($subQuery->getDQL()))
            ->setParameter($parameterName, $codes);
    }

    public function getDescription(string $resourceClass): array
    {
        return [
            self::DIAGNOSIS_FILTER_NAME => [
                'property' => self::DIAGNOSIS_FILTER_NAME,
                'type' => 'array',
                'required' => false,
                'description' => 'Фильтр пациентов по коду диагноза (активное лечение)',
            ],
        ];
    }
}

```

api/src/Ibs/Context/TreatmentTherapy/Filter/PatientDrugFilter.php
<?php

```

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Filter;

use ApiPlatform\Doctrine\Orm\Filter\AbstractFilter;
use ApiPlatform\Doctrine\Orm\Util\QueryNameGeneratorInterface;
use ApiPlatform\Metadata\Operation;
use Ibs\Context\TreatmentTherapy\Entity\Treatment;
use Doctrine\ORM\QueryBuilder;
use Doctrine\Persistence\ManagerRegistry;
use Psr\Log\LoggerInterface;
use Symfony\Component\Serializer\NameConverter\NameConverterInterface;

```

```

final class PatientDrugFilter extends AbstractFilter
{
    public const DRUG_FILTER_NAME = 'drug';

    public function __construct(
        ManagerRegistry $managerRegistry,
        ?LoggerInterface $logger = null,
        ?array $properties = null,
        ?NameConverterInterface $nameConverter = null
    ) {
        parent::__construct($managerRegistry, $logger, $properties, $nameConverter);
    }

    protected function filterProperty(
        string $property,
        $value,
        QueryBuilder $queryBuilder,
        QueryNameGeneratorInterface $queryNameGenerator,
        string $resourceClass,
        ?Operation $operation = null,
        array $context = []
    ): void {
        if ($property !== self::DRUG_FILTER_NAME) {
            return;
        }

        $parameterName = $queryNameGenerator->generateParameterName(self::DRUG_FILTER_NAME);
        $entityManager = $this->managerRegistry->getManager();

        $subQuery = $entityManager->createQueryBuilder()
            ->select('1')
            ->from(Treatment::class, 't_drug')
            ->where('t_drug.patient = o.id')
            ->andWhere('t_drug.begDt = (
                SELECT MAX(t2_drug.begDt)
                FROM Ibs\Context\TreatmentTherapy\Entity\Treatment t2_drug
                WHERE t2_drug.patient = o.id
            )')
            ->andWhere('t_drug.drug = :'. $parameterName)
            ->andWhere('t_drug.realEndDt IS NULL');

        $queryBuilder->andWhere($queryBuilder->expr()->exists($subQuery->getDQL()))
            ->setParameter($parameterName, $value);
    }

    public function getDescription(string $resourceClass): array

```

```

        {
            return [
                self::DRUG_FILTER_NAME => [
                    'property' => self::DRUG_FILTER_NAME,
                    'type' => 'int',
                    'required' => false,
                    'description' => 'Фильтр пациентов по конкретном
у препарату (текущее лечение)',
                ],
            ];
        }
    }
}
api/src/Ibs/Context/TreatmentTherapy/Filter/PatientGroupFilter.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Filter;

use ApiPlatform\Doctrine\Orm\Filter\AbstractFilter;
use ApiPlatform\Doctrine\Orm\Util\QueryNameGeneratorInterface;
use ApiPlatform\Metadata\Operation;
use Ibs\Context\TreatmentTherapy\Entity\Treatment;
use Doctrine\ORM\QueryBuilder;
use Doctrine\Persistence\ManagerRegistry;
use Psr\Log\LoggerInterface;
use Symfony\Component\Serializer\NameConverter\NameConverterInte
rface;

final class PatientGroupFilter extends AbstractFilter
{
    public const DRUG_GROUP_FILTER_NAME = 'drugGroup';

    public function __construct(
        ManagerRegistry $managerRegistry,
        ?LoggerInterface $logger = null,
        ?array $properties = null,
        ?NameConverterInterface $nameConverter = null
    )
    {
        parent::__construct($managerRegistry, $logger, $properti
es, $nameConverter);
    }

    protected function filterProperty(
        string $property,
        $value,
        QueryBuilder $queryBuilder,
        QueryNameGeneratorInterface $queryNameGenerator,
        string $resourceClass,

```

```

        ?Operation $operation = null,
        array $context = []
    ): void {
        if ($property !== self::DRUG_GROUP_FILTER_NAME) {
            return;
        }

        $parameterName = $queryNameGenerator->generateParameterName(
            self::DRUG_GROUP_FILTER_NAME);

        $entityManager = $this->managerRegistry->getManager();

        $subQuery = $entityManager->createQueryBuilder()
            ->select('1')
            ->from(Treatment::class, 't_group')
            ->where('t_group.patient = o.id')
            ->andWhere('t_group.begDt = (
                SELECT MAX(t2_group.begDt)
                FROM Ibs\Context\TreatmentTherapy\Entity\Treatment t
                WHERE t2_group.patient = o.id
            )')
            ->andWhere('t_group.drug IN (
                SELECT d.id
                FROM Ibs\Context\TreatmentTherapy\Entity\Drug d
                WHERE d.group = : ' . $parameterName . '
            )');

        $queryBuilder->andWhere($queryBuilder->expr()->exists($subQuery->getDQL()))
            ->setParameter($parameterName, $value);
    }

    public function getDescription(string $resourceClass): array
    {
        return [
            self::DRUG_GROUP_FILTER_NAME => [
                'property' => self::DRUG_GROUP_FILTER_NAME,
                'type' => 'int',
                'required' => false,
                'description' => 'Фильтр пациентов по группе препаратов',
            ],
        ];
    }
}

api/src/Ibs/Context/TreatmentTherapy/Entity/Treatment.php
<?php

declare(strict_types=1);

```

```

namespace Ibs\Context\TreatmentTherapy\Entity;

use ApiPlatform\Doctrine\Orm\Filter\OrderFilter;
use ApiPlatform\Doctrine\Orm\Filter\SearchFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Delete;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use ApiPlatform\Metadata\Patch;
use ApiPlatform\Metadata\Post;
use Ibs\Context\PatientManagement\Entity\Patient;
use Ibs\Context\TreatmentTherapy\Filter\ActiveTreatmentFilter;
use Doctrine\ORM\Mapping as ORM;
use Doctrine\ORM\Mapping\PrePersist;
use Doctrine\ORM\Mapping\PreUpdate;
use Symfony\Component\Validator\Constraints as Assert;

#[ApiResource(
    operations: [
        new GetCollection(
            name: 'get_all_without_pagination',
            uriTemplate: '/treatments/all',
            paginationEnabled: false,
        ),
        new GetCollection(),
        new Get(),
        new Post(),
        new Patch(),
        new Delete(),
    ],
)]
#[ORM\Entity]
#[ORM\Table(name: 'treatments')]
#[ORM\Index(name: 'idx_treatment_patient_id', columns: ['patient_id'])]
#[ORM\Index(name: 'idx_treatment_patient_beg_dt', columns: ['patient_id', 'beg_dt'])]
#[ORM\Index(name: 'idx_treatment_drug_id', columns: ['drug_id'])]
#[ApiFilter(ActiveTreatmentFilter::class)]
#[ApiFilter(SearchFilter::class, properties: [
    'patient' => 'exact',
])]
#[ApiFilter(OrderFilter::class, properties: ['begDt', 'realEndDt'])]
class Treatment
{
    #[ORM\Id]

```

```

#[ORM\GeneratedValue(strategy: 'IDENTITY')]
#[ORM\Column(type: 'integer', nullable: true)]
private ?int $id = null;

#[ORM\Column(type: 'datetime', nullable: true)]
private ?\DateTimeInterface $modDt = null;

#[ORM\Column(type: 'integer', unique: true, nullable: true)]
private ?int $code = null;

#[Assert\NotBlank(message: 'treatment.diagnosis.not_blank')]
#[ORM\Column(type: 'text', nullable: false)]
private string $diagnosis;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $comorbidities = null;

#[Assert\GreaterThan(0, message: 'treatment.mnoFrom.greater_
than_zero')]
#[ORM\Column(type: 'float', nullable: false)]
private float $mnoFrom;

#[Assert\GreaterThan(0, message: 'treatment.mnoTo.greater_th
an_zero')]
#[ORM\Column(type: 'float', nullable: false)]
private float $mnoTo;

#[Assert\Type(\DateTimeInterface::class, message: 'treatment
.begDt.type')]
#[ORM\Column(type: 'datetime', nullable: false)]
private \DateTimeInterface $begDt;

#[ORM\Column(type: 'datetime', nullable: true)]
private ?\DateTimeInterface $planEndDt = null;

#[ORM\Column(type: 'datetime', nullable: true)]
private ?\DateTimeInterface $realEndDt = null;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $stoppingReason = null;

#[ORM\ManyToOne(targetEntity: Patient::class)]
#[ORM\JoinColumn(name: 'patient_id', referencedColumnName: '
id', nullable: true)]
private ?Patient $patient = null;

#[Assert\NotBlank(message: 'treatment.drug.not_blank')]
#[ORM\ManyToOne(targetEntity: Drug::class)]
#[ORM\JoinColumn(name: 'drug_id', referencedColumnName: 'id'
, nullable: true)]

```

```

private ?Drug $drug = null;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $comment = null;

#[ORM\Column(type: 'integer', nullable: false, options: ['default' => 0])]
private int $hemorrhages = 0;

#[ORM\Column(type: 'integer', nullable: false, options: ['default' => 0])]
private int $flags = 0;

#[Assert\NotBlank(message: 'treatment.diagnosisCode.not_blank')]
#[Assert\Length(max: 255, maxMessage: 'treatment.diagnosisCode.length')]
#[ORM\Column(name: 'diagnosis_code', type: 'string', length: 255, nullable: true)]
private ?string $diagnosisCode = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $pin = null;

#[ORM\ManyToOne(targetEntity: Mkb10::class)]
#[ORM\JoinColumn(name: 'mkb10_id', referencedColumnName: 'id', nullable: true)]
private ?Mkb10 $mkb10 = null;

public function getId(): ?int
{
    return $this->id;
}

public function getModDt(): ?\DateTimeInterface
{
    return $this->modDt;
}

public function setModDt(?\DateTimeInterface $modDt): self
{
    $this->modDt = $modDt;
    return $this;
}

public function getCode(): ?int
{
    return $this->code;
}

```

```

public function setCode(?int $code): self
{
    $this->code = $code;
    return $this;
}

public function getDiagnosis(): string
{
    return $this->diagnosis;
}

public function setDiagnosis(string $diagnosis): self
{
    $this->diagnosis = $diagnosis;
    return $this;
}

public function getComorbidities(): ?string
{
    return $this->comorbidities;
}

public function setComorbidities(?string $comorbidities): self
{
    $this->comorbidities = $comorbidities;
    return $this;
}

public function getMnoFrom(): float
{
    return $this->mnoFrom;
}

public function setMnoFrom(float $mnoFrom): self
{
    $this->mnoFrom = $mnoFrom;
    return $this;
}

public function getMnoTo(): float
{
    return $this->mnoTo;
}

public function setMnoTo(float $mnoTo): self
{
    $this->mnoTo = $mnoTo;
    return $this;
}

```

```

public function getBegDt(): \DateTimeInterface
{
    return $this->begDt;
}

public function setBegDt(\DateTimeInterface $begDt): self
{
    $this->begDt = $begDt;
    return $this;
}

public function getPlanEndDt(): ?\DateTimeInterface
{
    return $this->planEndDt;
}

public function setPlanEndDt(?\DateTimeInterface $planEndDt)
: self
{
    $this->planEndDt = $planEndDt;
    return $this;
}

public function getRealEndDt(): ?\DateTimeInterface
{
    return $this->realEndDt;
}

public function setRealEndDt(?\DateTimeInterface $realEndDt)
: self
{
    $this->realEndDt = $realEndDt;
    return $this;
}

public function getStoppingReason(): ?string
{
    return $this->stoppingReason;
}

public function setStoppingReason(?string $stoppingReason):
self
{
    $this->stoppingReason = $stoppingReason;
    return $this;
}

public function getPatient(): ?Patient
{

```

```

        return $this->patient;
    }

    public function setPatient(?Patient $patient): self
    {
        $this->patient = $patient;
        return $this;
    }

    public function getDrug(): ?Drug
    {
        return $this->drug;
    }

    public function setDrug(?Drug $drug): self
    {
        $this->drug = $drug;
        return $this;
    }

    public function getComment(): ?string
    {
        return $this->comment;
    }

    public function setComment(?string $comment): self
    {
        $this->comment = $comment;
        return $this;
    }

    public function getHemorrhages(): int
    {
        return $this->hemorrhages;
    }

    public function setHemorrhages(int $hemorrhages): self
    {
        $this->hemorrhages = $hemorrhages;
        return $this;
    }

    public function getFlags(): int
    {
        return $this->flags;
    }

    public function setFlags(int $flags): self
    {
        $this->flags = $flags;
    }

```

```

        return $this;
    }

    public function getDiagnosisCode(): ?string
    {
        return $this->diagnosisCode;
    }

    public function setDiagnosisCode(?string $diagnosisCode): self
    {
        $this->diagnosisCode = $diagnosisCode;
        return $this;
    }

    public function getPin(): ?int
    {
        return $this->pin;
    }

    public function setPin(?int $pin): self
    {
        $this->pin = $pin;
        return $this;
    }

    public function getMkb10(): ?Mkb10
    {
        return $this->mkb10;
    }

    public function setMkb10(?Mkb10 $mkb10): self
    {
        $this->mkb10 = $mkb10;
        return $this;
    }

    #[Assert\Callback]
    public function validate(\Symfony\Component\Validator\Context\ExecutionContextInterface $context): void
    {
        if ($this->mnoFrom >= $this->mnoTo) {
            $context->buildViolation('treatment.mno_range.invalid')
                ->atPath('mnoFrom')
                ->addViolation();
        }

        if ($this->planEndDt !== null && $this->begDt !== null)
    {

```

```

        if ($this->planEndDt < $this->begDt) {
            $context->buildViolation('treatment.plan_end_dt.
invalid')
                ->atPath('planEndDt')
                ->addViolation();
        }
    }
}

#[PrePersist]
#[PreUpdate]
public function setUpdatedAtValue(): void
{
    $this->modDt = new \DateTime();
}
}
api/src/Ibs/Context/PatientManagement/Entity/Patient.php
<?php

declare(strict_types=1);

namespace Ibs\Context\PatientManagement\Entity;

use ApiPlatform\Doctrine\Orm\Filter\OrderFilter;
use ApiPlatform\Doctrine\Orm\Filter\SearchFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use Ibs\Context\TreatmentTherapy\Filter\PatientDiagnosisFilter;
use Ibs\Context\TreatmentTherapy\Filter\PatientDrugFilter;
use Ibs\Context\TreatmentTherapy\Filter\PatientGroupFilter;
use Doctrine\ORM\Mapping as ORM;
use Doctrine\ORM\Mapping\PrePersist;
use Doctrine\ORM\Mapping\PreUpdate;
use Symfony\Component\Validator\Constraints as Assert;

#[ApiResource]
#[ApiFilter(SearchFilter::class, properties: [
    'lastname' => 'ipartial',
    'hospital' => 'exact'
])]
#[ApiFilter(OrderFilter::class, properties: ['lastname'])]
#[ApiFilter(PatientGroupFilter::class, properties: [PatientGroup
    Filter::DRUG_GROUP_FILTER_NAME])]
#[ApiFilter(PatientDrugFilter::class, properties: ['drug'])]
#[ApiFilter(PatientDiagnosisFilter::class, properties: ['diagnos
    isCode'])]
#[ORM\Entity]
#[ORM\Table(name: 'patients')]
class Patient
{

```

```

#[ORM\Id]
#[ORM\GeneratedValue(strategy: 'IDENTITY')]
#[ORM\Column(type: 'integer', nullable: true)]
private ?int $id = null;

#[ORM\Column(type: 'datetime', nullable: true)]
private ?\DateTimeInterface $modDt = null;

#[Assert\NotBlank(message: 'patient.firstname.not_blank')]
#[Assert\Length(
    min: 2,
    max: 255,
    minMessage: 'patient.firstname.length',
    maxMessage: 'patient.firstname.length'
)]
#[ORM\Column(type: 'text', nullable: false)]
private string $firstname;

#[Assert\Length(
    min: 2,
    max: 255,
    minMessage: 'patient.secondName.length',
    maxMessage: 'patient.secondName.length'
)]
#[ORM\Column(type: 'text', nullable: true)]
private ?string $secondName = null;

#[Assert\NotBlank(message: 'patient.lastname.not_blank')]
#[Assert\Length(
    min: 2,
    max: 255,
    minMessage: 'patient.lastname.length',
    maxMessage: 'patient.lastname.length'
)]
#[ORM\Column(type: 'text', nullable: false)]
private string $lastname;

#[Assert\NotBlank(message: 'patient.birthday.not_blank')]
#[Assert\Type(\DateTimeInterface::class, message: 'patient.birthday.type')]
#[Assert\LessThanOrEqual(value: 'today', message: 'patient.birthday.max')]
#[Assert\GreaterThanOrEqual(value: '-120 years', message: 'patient.birthday.min')]
#[ORM\Column(type: 'datetime', nullable: false)]
private \DateTimeInterface $birthday;

#[Assert\NotBlank(message: 'patient.sex.not_blank')]
#[Assert\Type(type: 'integer', message: 'patient.sex.type')]
#[ORM\Column(type: 'integer', nullable: false, options: ['de

```

```

fault' => 0]])
private int $sex = 0;

#[Assert\NotBlank(message: 'patient.smsPhone.not_blank')]
#[Assert\Regex(pattern: '/^8\(\d{3}\)\d{3}-\d{2}-\d{2}$/', message: 'patient.smsPhone.regex')]
#[ORM\Column(type: 'text', nullable: false)]
private string $smsPhone;

#[Assert\NotBlank(message: 'patient.address.not_blank')]
#[ORM\Column(type: 'text', nullable: true)]
private ?string $address = null;

#[Assert\NotBlank(message: 'patient.passport.not_blank')]
#[Assert\Regex(pattern: '/^\d{4} \d{6}$/', message: 'patient.passport.regex')]
#[ORM\Column(type: 'text', nullable: true)]
private ?string $passport = null;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $healthInsurance = null;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $comment = null;

#[Assert\NotBlank(message: 'patient.snils.not_blank')]
#[Assert\Regex(pattern: '/^\d{3}-\d{3}-\d{3} \d{2}$/', message: 'patient.snils.regex')]
#[ORM\Column(type: 'text', nullable: true)]
private ?string $snils = null;

#[ORM\Column(type: 'string', length: 180, nullable: true)]
#[Assert\Email(message: 'patient.email.invalid')]
private ?string $email = null;

#[Assert\NotBlank(message: 'patient.hospital.not_blank')]
#[ORM\ManyToOne(targetEntity: Hospital::class)]
#[ORM\JoinColumn(name: 'hospital_id', referencedColumnName: 'id', nullable: true)]
private ?Hospital $hospital = null;

public function getId(): ?int
{
    return $this->id;
}

public function getModDt(): ?\DateTimeInterface
{
    return $this->modDt;
}

```

```

public function setModDt(?DateTimeInterface $modDt): self
{
    $this->modDt = $modDt;
    return $this;
}

public function getFirstname(): string
{
    return $this->firstname;
}

public function setFirstname(string $firstname): self
{
    $this->firstname = $firstname;
    return $this;
}

public function getSecondName(): ?string
{
    return $this->secondName;
}

public function setSecondName(?string $secondName): self
{
    $this->secondName = $secondName;
    return $this;
}

public function getLastname(): string
{
    return $this->lastname;
}

public function setLastname(string $lastname): self
{
    $this->lastname = $lastname;
    return $this;
}

public function getBirthday(): DateTimeInterface
{
    return $this->birthday;
}

public function setBirthday(DateTimeInterface $birthday): self
{
    $this->birthday = $birthday;
    return $this;
}

```

```

}

public function getSex(): int
{
    return $this->sex;
}

public function setSex(int $sex): self
{
    $this->sex = $sex;
    return $this;
}

public function getSmsPhone(): string
{
    return $this->smsPhone;
}

public function setSmsPhone(string $smsPhone): self
{
    $this->smsPhone = $smsPhone;
    return $this;
}

public function getAddress(): ?string
{
    return $this->address;
}

public function setAddress(?string $address): self
{
    $this->address = $address;
    return $this;
}

public function getPassport(): ?string
{
    return $this->passport;
}

public function setPassport(?string $passport): self
{
    $this->passport = $passport;
    return $this;
}

public function getHealthInsurance(): ?string
{
    return $this->healthInsurance;
}

```

```

public function setHealthInsurance(?string $healthInsurance)
: self
{
    $this->healthInsurance = $healthInsurance;
    return $this;
}

public function getComment(): ?string
{
    return $this->comment;
}

public function setComment(?string $comment): self
{
    $this->comment = $comment;
    return $this;
}

public function getSnils(): ?string
{
    return $this->snils;
}

public function setSnils(?string $snils): self
{
    $this->snils = $snils;
    return $this;
}

public function getHospital(): ?Hospital
{
    return $this->hospital;
}

public function setHospital(?Hospital $hospital): self
{
    $this->hospital = $hospital;
    return $this;
}

public function getEmail(): ?string
{
    return $this->email;
}

public function setEmail(?string $email): self
{
    $this->email = $email;
    return $this;
}

```

```

    }

    #[PrePersist]
    #[PreUpdate]
    public function setUpdatedAtValue(): void
    {
        $this->modDt = new \DateTime();
    }
}
api/src/Ibs/Context/TreatmentTherapy/Entity/Appointment.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Entity;

use ApiPlatform\Doctrine\Orm\FILTER\OrderFilter;
use ApiPlatform\Doctrine\Orm\FILTER\SearchFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Delete;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use ApiPlatform\Metadata\Patch;
use ApiPlatform\Metadata\Post;
use Ibs\Context\Communication\Entity\SmsOut;
use Ibs\Context\TreatmentTherapy\State\AppointmentSaveProcessor;
use Doctrine\ORM\Mapping as ORM;
use Doctrine\ORM\Mapping\PrePersist;
use Doctrine\ORM\Mapping\PreUpdate;

#[ApiResource(
    operations: [
        new GetCollection(),
        new Post(processor: AppointmentSaveProcessor::class),
        new Get(),
        new Patch(),
        new Delete(),
    ],
)]
#[ORM\Entity]
#[ORM\Table(name: 'appointments')]
#[ApiFilter(SearchFilter::class, properties: ['treatment' => 'exact'])]
#[ApiFilter(OrderFilter::class, properties: ['appointmentDt'])]
class Appointment
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer', nullable: true)]

```

```

private ?int $id = null;

#[ORM\Column(type: 'datetime', nullable: true)]
private ?\DateTimeInterface $modDt = null;

#[ORM\Column(type: 'datetime', nullable: false)]
private \DateTimeInterface $appointmentDt;

#[ORM\Column(type: 'datetime', nullable: true)]
private ?\DateTimeInterface $creationDt = null;

#[ORM\Column(type: 'float', nullable: false)]
private float $doze;

#[ORM\Column(type: 'text', nullable: false)]
private string $doctorName;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $comment = null;

#[ORM\Column(type: 'integer', nullable: false, options: ['default' => -1])]
private int $doze2 = -1;

#[ORM\ManyToOne(targetEntity: Treatment::class)]
#[ORM\JoinColumn(name: 'treatment_id', referencedColumnName: 'id', nullable: true)]
private ?Treatment $treatment = null;

#[ORM\ManyToOne(targetEntity: SmsOut::class)]
#[ORM\JoinColumn(name: 'sms_id', referencedColumnName: 'id', nullable: true)]
private ?SmsOut $sms = null;

#[ORM\ManyToOne(targetEntity: Drug::class)]
#[ORM\JoinColumn(name: 'drug_id', referencedColumnName: 'id', nullable: true)]
private ?Drug $drug = null;

public function getId(): ?int
{
    return $this->id;
}

public function getModDt(): ?\DateTimeInterface
{
    return $this->modDt;
}

public function setModDt(?\DateTimeInterface $modDt): self

```

```

{
    $this->modDt = $modDt;
    return $this;
}

public function getAppointmentDt(): \DateTimeInterface
{
    return $this->appointmentDt;
}

public function setAppointmentDt(\DateTimeInterface $appointmentDt): self
{
    $this->appointmentDt = $appointmentDt;
    return $this;
}

public function getCreationDt(): ?\DateTimeInterface
{
    return $this->creationDt;
}

public function setCreationDt(?\DateTimeInterface $creationDt): self
{
    $this->creationDt = $creationDt;
    return $this;
}

public function getDoze(): float
{
    return $this->doze;
}

public function setDoze(float $doze): self
{
    $this->doze = $doze;
    return $this;
}

public function getDoctorName(): string
{
    return $this->doctorName;
}

public function setDoctorName(string $doctorName): self
{
    $this->doctorName = $doctorName;
    return $this;
}

```

```

public function getComment(): ?string
{
    return $this->comment;
}

public function setComment(?string $comment): self
{
    $this->comment = $comment;
    return $this;
}

public function getDoze2(): int
{
    return $this->doze2;
}

public function setDoze2(int $doze2): self
{
    $this->doze2 = $doze2;
    return $this;
}

public function getTreatment(): ?Treatment
{
    return $this->treatment;
}

public function setTreatment(?Treatment $treatment): self
{
    $this->treatment = $treatment;
    return $this;
}

public function getSms(): ?SmsOut
{
    return $this->sms;
}

public function setSms(?SmsOut $sms): self
{
    $this->sms = $sms;
    return $this;
}

public function getDrug(): ?Drug
{
    return $this->drug;
}

```

```

    public function setDrug(?Drug $drug): self
    {
        $this->drug = $drug;
        return $this;
    }

    #[PrePersist]
    public function setCreatedAtValue(): void
    {
        if ($this->creationDt === null) {
            $this->creationDt = new \DateTime();
        }
        $this->modDt = new \DateTime();
    }

    #[PreUpdate]
    public function setUpdatedAtValue(): void
    {
        $this->modDt = new \DateTime();
    }
}
api/src/Ibs/Context/TreatmentTherapy/Entity/TestHistory.php
<?php

```

```

declare(strict_types=1);

```

```

namespace Ibs\Context\TreatmentTherapy\Entity;

```

```

use ApiPlatform\Doctrine\Orm\FILTER\OrderFilter;
use ApiPlatform\Doctrine\Orm\FILTER/SearchFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Delete;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use ApiPlatform\Metadata\Patch;
use ApiPlatform\Metadata\Post;
use Ibs\Context\TreatmentTherapy\State\TestHistoryLatestProvider
;
use Doctrine\ORM\Mapping as ORM;
use Doctrine\ORM\Mapping\PrePersist;
use Doctrine\ORM\Mapping\PreUpdate;
use Symfony\Component\Validator\Constraints as Assert;

```

```

#[ApiResource(
    operations: [
        new GetCollection(
            name: 'latest_test_histories',
            uriTemplate: '/test_histories/latest',
            provider: TestHistoryLatestProvider::class,

```

```

        paginationEnabled: false,
    ),
    new GetCollection(),
    new Get(),
    new Post(),
    new Patch(),
    new Delete(),
],
)]
#[ORM\Entity]
#[ORM\Table(name: 'test_history')]
#[ApiFilter(SearchFilter::class, properties: ['treatment' => 'exact'])]
#[ApiFilter(OrderFilter::class, properties: ['creationDt'])]
class TestHistory
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer', nullable: true)]
    private ?int $id = null;

    #[ORM\Column(type: 'datetime', nullable: true)]
    private ?\DateTimeInterface $modDt = null;

    #[ORM\Column(type: 'integer', nullable: true)]
    private ?int $smsId = null;

    #[ORM\Column(type: 'datetime', nullable: true)]
    private ?\DateTimeInterface $creationDt = null;

    #[ORM\Column(type: 'float', nullable: false)]
    #[Assert\NotBlank(message: 'Значение МНО обязательно')]
    #[Assert\GreaterThanOrEqual(value: 0.8, message: 'МНО должно быть не менее 0.8')]
    #[Assert\LessThanOrEqual(value: 10.0, message: 'МНО должно быть не более 10.0')]
    private float $mno = 0.0;

    #[ORM\ManyToOne(targetEntity: Treatment::class, cascade: ['persist'])]
    #[ORM\JoinColumn(name: 'treatment_id', referencedColumnName: 'id', nullable: true)]
    private ?Treatment $treatment = null;

    #[ORM\ManyToOne(targetEntity: Drug::class, cascade: ['persist'])]
    #[ORM\JoinColumn(name: 'drug_id', referencedColumnName: 'id', nullable: true)]
    private ?Drug $drug = null;
}

```

```

#[ORM\Column(type: 'float', nullable: false)]
#[Assert\NotBlank(message: 'Доза обязательна')]
#[Assert\Positive(message: 'Доза должна быть положительной')]
]
private float $doze = 0.0;

#[ORM\Column(type: 'integer', nullable: false, options: ['default' => -1])]
private int $doze2 = -1;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $comment = null;

// ----- lifecycle callbacks -----

#[PrePersist]
public function setCreatedAtValue(): void
{
    if ($this->creationDt === null) {
        $this->creationDt = new \DateTime();
    }
    if ($this->modDt === null) {
        $this->modDt = new \DateTime();
    }
    // Гарантируем, что doze2 всегда -1, если не задан явно
    if ($this->doze2 === 0 || $this->doze2 === null) {
        $this->doze2 = -1;
    }
}

#[PreUpdate]
public function setUpdatedAtValue(): void
{
    $this->modDt = new \DateTime();
}

public function getId(): ?int
{
    return $this->id;
}

public function getModDt(): ?\DateTimeInterface
{
    return $this->modDt;
}

public function setModDt(?\DateTimeInterface $modDt): self
{
    $this->modDt = $modDt;
    return $this;
}

```

```

}

public function getSmsId(): ?int
{
    return $this->smsId;
}

public function setSmsId(?int $smsId): self
{
    $this->smsId = $smsId;
    return $this;
}

public function getCreationDt(): ?\DateTimeInterface
{
    return $this->creationDt;
}

public function setCreationDt(?\DateTimeInterface $creationDt): self
{
    $this->creationDt = $creationDt;
    return $this;
}

public function getMno(): float
{
    return $this->mno;
}

public function setMno(float $mno): self
{
    $this->mno = $mno;
    return $this;
}

public function getTreatment(): ?Treatment
{
    return $this->treatment;
}

public function setTreatment(?Treatment $treatment): self
{
    $this->treatment = $treatment;
    return $this;
}

public function getDrug(): ?Drug
{
    return $this->drug;
}

```

```

    }

    public function setDrug(?Drug $drug): self
    {
        $this->drug = $drug;
        return $this;
    }

    public function getDoze(): float
    {
        return $this->doze;
    }

    public function setDoze(float $doze): self
    {
        $this->doze = $doze;
        return $this;
    }

    public function getDoze2(): int
    {
        return $this->doze2;
    }

    public function setDoze2(int $doze2): self
    {
        $this->doze2 = $doze2;
        return $this;
    }

    public function getComment(): ?string
    {
        return $this->comment;
    }

    public function setComment(?string $comment): self
    {
        $this->comment = $comment;
        return $this;
    }
}
api/src/Ibs/Context/SecurityIdentity/Entity/User.php
<?php

declare(strict_types=1);

namespace Ibs\Context\SecurityIdentity\Entity;

use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;

```

```

use ApiPlatform\Metadata\GetCollection;
use ApiPlatform\Metadata\Post;
use ApiPlatform\Metadata\Put;
use ApiPlatform\Metadata\Delete;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
use Symfony\Component\Security\Core\User\UserInterface;
use Symfony\Component\Serializer\Annotation\Groups;

#[ApiResource(
    operations: [
        new GetCollection(
            normalizationContext: ['groups' => ['user:read']],
            security: "is_granted('ROLE_ADMIN')"
        ),
        new Get(
            normalizationContext: ['groups' => ['user:read']],
            security: "is_granted('ROLE_ADMIN') or object == use
r"
        ),
        new Post(
            denormalizationContext: ['groups' => ['user:write']]
            ,
            security: "is_granted('ROLE_ADMIN')"
        ),
        new Put(
            denormalizationContext: ['groups' => ['user:write']]
            ,
            security: "is_granted('ROLE_ADMIN') or object == use
r"
        ),
        new Delete(
            security: "is_granted('ROLE_ADMIN')"
        ),
    ]
)]

#[ORM\Entity]
#[ORM\Table(name: 'users')]
#[ORM\Index(name: 'idx_users_medpers', columns: ['medical_personnel_id'])]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer', nullable: true)]
    #[Groups(['user:read'])]
    private ?int $id = null;

```

```

#[ORM\Column(type: 'text', nullable: true)]
#[Groups(['user:read', 'user:write'])]
private ?string $login = null;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $password = null;

#[ORM\Column(type: 'text', nullable: true)]
#[Groups(['user:read', 'user:write'])]
private ?string $userName = null;

#[ORM\Column(type: 'json', nullable: false, options: ['default' => '[]'])]
#[Groups(['user:read'])]
private array $roles = [];

#[ORM\Column(type: 'text', nullable: true)]
private ?string $comment = null;

#[ORM\ManyToOne(targetEntity: MedicalPersonnel::class)]
#[ORM\JoinColumn(name: 'medical_personnel_id', referencedColumnName: 'id', nullable: true)]
#[Groups(['user:read'])]
private ?MedicalPersonnel $medicalPersonnel = null;

public function getId(): ?int
{
    return $this->id;
}

public function getLogin(): ?string
{
    return $this->login;
}

public function setLogin(?string $login): self
{
    $this->login = $login;
    return $this;
}

public function getPassword(): ?string
{
    return $this->password;
}

public function setPassword(?string $password): self
{
    $this->password = $password;
    return $this;
}

```

```

}

public function getUsername(): ?string
{
    return $this->userName;
}

public function setUsername(?string $userName): self
{
    $this->userName = $userName;
    return $this;
}

public function getRoles(): array
{
    $roles = $this->roles;
    if (!in_array('ROLE_USER', $roles, true)) {
        $roles[] = 'ROLE_USER';
    }
    return array_unique($roles);
}

public function setRoles(array $roles): self
{
    $this->roles = $roles;
    return $this;
}

public function getComment(): ?string
{
    return $this->comment;
}

public function setComment(?string $comment): self
{
    $this->comment = $comment;
    return $this;
}

public function getMedicalPersonnel(): ?MedicalPersonnel
{
    return $this->medicalPersonnel;
}

public function setMedicalPersonnel(?MedicalPersonnel $medicalPersonnel): self
{
    $this->medicalPersonnel = $medicalPersonnel;
    return $this;
}

```

```

/**
 * @see UserInterface
 */
public function getUserIdfier(): string
{
    return (string) $this->login;
}

/**
 * @deprecated since Symfony 7.3, use __serialize() instead
 */
#[\Deprecated]
public function eraseCredentials(): void
{
    // Пароль не сохраняется в сессии благодаря __serialize(
)
}

public function __serialize(): array
{
    return [
        'id' => $this->id,
        'login' => $this->login,
        'userName' => $this->userName,
        'roles' => $this->roles,
        'medicalPersonnel' => $this->medicalPersonnel,
        'comment' => $this->comment,
    ];
}

public function __unserialize(array $data): void
{
    $this->id = $data['id'] ?? null;
    $this->login = $data['login'] ?? null;
    $this->userName = $data['userName'] ?? null;
    $this->roles = $data['roles'] ?? [];
    $this->medicalPersonnel = $data['medicalPersonnel'] ?? null;
    $this->comment = $data['comment'] ?? null;
}
}
api/src/Ibs/Context/SecurityIdentity/Entity/MedicalPersonnel.php
<?php

declare(strict_types=1);

namespace Ibs\Context\SecurityIdentity\Entity;

use ApiPlatform\Metadata\ApiResource;

```

```

use Ibs\Context\PatientManagement\Entity\Hospital;
use Doctrine\ORM\Mapping as ORM;

#[ApiResponse]
#[ORM\Entity]
#[ORM\Table(name: 'medical_personnel')]
class MedicalPersonnel
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer', nullable: true)]
    private ?int $id = null;

    #[ORM\Column(type: 'datetime', nullable: true)]
    private ?\DateTimeInterface $modDt = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $name = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $post = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $address = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $comment = null;

    #[ORM\ManyToOne(targetEntity: Hospital::class)]
    #[ORM\JoinColumn(name: 'hospital_id', referencedColumnName: 'id')]
    private ?Hospital $hospital = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getModDt(): ?\DateTimeInterface
    {
        return $this->modDt;
    }

    public function setModDt(?\DateTimeInterface $modDt): self
    {
        $this->modDt = $modDt;
        return $this;
    }

    public function getName(): ?string

```

```

{
    return $this->name;
}

public function setName(?string $name): self
{
    $this->name = $name;
    return $this;
}

public function getPost(): ?string
{
    return $this->post;
}

public function setPost(?string $post): self
{
    $this->post = $post;
    return $this;
}

public function getAddress(): ?string
{
    return $this->address;
}

public function setAddress(?string $address): self
{
    $this->address = $address;
    return $this;
}

public function getComment(): ?string
{
    return $this->comment;
}

public function setComment(?string $comment): self
{
    $this->comment = $comment;
    return $this;
}

public function getHospital(): ?Hospital
{
    return $this->hospital;
}

public function setHospital(?Hospital $hospital): self
{

```

```

        $this->hospital = $hospital;
        return $this;
    }}
api/src/Ibs/Context/PatientManagement/Entity/Hospital.php
<?php

declare(strict_types=1);

namespace Ibs\Context\PatientManagement\Entity;

use ApiPlatform\Doctrine\Orm\Filter\OrderFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use Doctrine\ORM\Mapping as ORM;

#[ApiResource]
#[ORM\Entity]
#[ORM\Table(name: 'hospitals')]
#[ApiFilter(OrderFilter::class, properties: ['name'])]
class Hospital
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer', nullable: true)]
    private ?int $id = null;

    #[ORM\Column(type: 'datetime', nullable: true)]
    private ?\DateTimeInterface $modDt = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $name = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $region = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $smsPhone = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $address = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $comment = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getModDt(): ?\DateTimeInterface

```

```

{
    return $this->modDt;
}

public function setModDt(?DateTimeInterface $modDt): self
{
    $this->modDt = $modDt;
    return $this;
}

public function getName(): ?string
{
    return $this->name;
}

public function setName(?string $name): self
{
    $this->name = $name;
    return $this;
}

public function getRegion(): ?string
{
    return $this->region;
}

public function setRegion(?string $region): self
{
    $this->region = $region;
    return $this;
}

public function getSmsPhone(): ?string
{
    return $this->smsPhone;
}

public function setSmsPhone(?string $smsPhone): self
{
    $this->smsPhone = $smsPhone;
    return $this;
}

public function getAddress(): ?string
{
    return $this->address;
}

public function setAddress(?string $address): self
{

```

```

        $this->address = $address;
        return $this;
    }

    public function getComment(): ?string
    {
        return $this->comment;
    }

    public function setComment(?string $comment): self
    {
        $this->comment = $comment;
        return $this;
    }
}

```

api/src/Ibs/Context/TreatmentTherapy/Entity/Drug.php

```

<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Entity;

use ApiPlatform\Metadata\ApiResource;
use Doctrine\ORM\Mapping as ORM;

#[ApiResource]
#[ORM\Entity]
#[ORM\Table(name: 'drugs')]
#[ORM\Index(name: 'idx_drug_group_id', columns: ['group_id'])]
class Drug
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer', nullable: true)]
    private ?int $id = null;

    #[ORM\Column(type: 'datetime', nullable: true)]
    private ?\DateTimeInterface $modDt = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $nominative = null;

    #[ORM\Column(type: 'text', nullable: true)]
    private ?string $genitive = null;

    #[ORM\ManyToOne(targetEntity: DrugGroup::class)]
    #[ORM\JoinColumn(name: 'group_id', referencedColumnName: 'id')]
    private ?DrugGroup $group = null;
}

```

```

public function getId(): ?int
{
    return $this->id;
}

public function getModDt(): ?\DateTimeInterface
{
    return $this->modDt;
}

public function setModDt(?\DateTimeInterface $modDt): self
{
    $this->modDt = $modDt;
    return $this;
}

public function getNominative(): ?string
{
    return $this->nominative;
}

public function setNominative(?string $nominative): self
{
    $this->nominative = $nominative;
    return $this;
}

public function getGenitive(): ?string
{
    return $this->genitive;
}

public function setGenitive(?string $genitive): self
{
    $this->genitive = $genitive;
    return $this;
}

public function getGroup(): ?DrugGroup
{
    return $this->group;
}

public function setGroup(?DrugGroup $group): void
{
    $this->group = $group;
}
}

```

api/src/Ibs/Context/TreatmentTherapy/Entity/DrugGroup.php
<?php

```

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Entity;

use ApiPlatform\Metadata\ApiResource;
use Doctrine\ORM\Mapping as ORM;

#[ApiResource]
#[ORM\Entity]
#[ORM\Table(name: 'drug_groups')]
class DrugGroup
{
    #[ORM\Id]
    #[ORM\GeneratedValue(strategy: 'IDENTITY')]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;

    #[ORM\Column(type: 'string', length: 50)]
    private string $name;

    #[ORM\Column(type: 'string', length: 255, nullable: true)]
    private ?string $fullName = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getName(): string
    {
        return $this->name;
    }

    public function setName(string $name): self
    {
        $this->name = $name;
        return $this;
    }

    public function getFullName(): ?string
    {
        return $this->fullName;
    }

    public function setFullName(?string $fullName): self
    {
        $this->fullName = $fullName;
        return $this;
    }
}

```

```

}
api/src/Ibs/Context/TreatmentTherapy/Entity/Mkb10.php
<?php

declare(strict_types=1);

namespace Ibs\Context\TreatmentTherapy\Entity;

use ApiPlatform\Doctrine\Orm\FILTER\SearchFilter;
use ApiPlatform\Metadata\ApiFilter;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use Ibs\Context\TreatmentTherapy\State\Mkb10PopularProvider;
use Ibs\Context\TreatmentTherapy\State\Mkb10SearchProvider;
use Doctrine\ORM\Mapping as ORM;

#[ApiResource(
    shortName: 'mkb10',
    operations: [
        new GetCollection(
            name: 'popular',
            uriTemplate: '/mkb10/popular',
            provider: Mkb10PopularProvider::class,
            paginationEnabled: false,
        ),
        new GetCollection(
            name: 'search',
            uriTemplate: '/mkb10/search',
            provider: Mkb10SearchProvider::class,
            paginationEnabled: false,
        ),
        new GetCollection(uriTemplate: '/mkb10'),
        new Get(),
    ],
    order: ['mkbCode' => 'ASC'],
    paginationEnabled: false,
)]
#[ORM\Entity]
#[ORM\Table(name: 'mkb10')]
#[ApiFilter(SearchFilter::class, properties: [
    'mkbCode' => 'ipartial',
    'mkbName' => 'ipartial',
])]
class Mkb10
{
    #[ORM\Id]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;

```

```

#[ORM\Column(type: 'string', length: 50, nullable: true)]
private ?string $recCode = null;

#[ORM\Column(type: 'string', length: 50, nullable: true)]
private ?string $mkbCode = null;

#[ORM\Column(type: 'text', nullable: true)]
private ?string $mkbName = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $idParent = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $addlCode = null;

#[ORM\Column(type: 'integer', nullable: true)]
private ?int $actual = null;

#[ORM\Column(type: 'string', length: 50, nullable: true)]
private ?string $date = null;

public function getId(): ?int
{
    return $this->id;
}

public function setId(int $id): self
{
    $this->id = $id;
    return $this;
}

public function getRecCode(): ?string
{
    return $this->recCode;
}

public function setRecCode(?string $recCode): self
{
    $this->recCode = $recCode;
    return $this;
}

public function getMkbCode(): ?string
{
    return $this->mkbCode;
}

public function setMkbCode(?string $mkbCode): self
{

```

```

        $this->mkbCode = $mkbCode;
        return $this;
    }

    public function getMkbName(): ?string
    {
        return $this->mkbName;
    }

    public function setMkbName(?string $mkbName): self
    {
        $this->mkbName = $mkbName;
        return $this;
    }

    public function getIdParent(): ?int
    {
        return $this->idParent;
    }

    public function setIdParent(?int $idParent): self
    {
        $this->idParent = $idParent;
        return $this;
    }

    public function getAddlCode(): ?int
    {
        return $this->addlCode;
    }

    public function setAddlCode(?int $addlCode): self
    {
        $this->addlCode = $addlCode;
        return $this;
    }

    public function getActual(): ?int
    {
        return $this->actual;
    }

    public function setActual(?int $actual): self
    {
        $this->actual = $actual;
        return $this;
    }

    public function getDate(): ?string
    {

```

```

        return $this->date;
    }

    public function setDate(?string $date): self
    {
        $this->date = $date;
        return $this;
    }
}
frontend/src/modules/shared/stores/drugGroupStore.js
import { defineStore } from 'pinia';
import apiClient from '@modules/shared/api/client';

export const useDrugGroupStore = defineStore('drugGroup', {
    state: () => ({
        drugGroups: [],
        loading: false,
        error: null,
    }),

    getters: {
        drugGroupOptions: (state) => {
            return [
                { value: '', label: 'Все категории' },
                ...state.drugGroups.map(g => ({ value: g.id, label: g.fullName || g.name }))
            ];
        },
    },

    actions: {
        async loadDrugGroups() {
            this.loading = true;
            this.error = null;
            try {
                const response = await apiClient.get('/drug_groups');
                const groups = response.data?.member ?? response.data;
                this.drugGroups = Array.isArray(groups) ? groups : [];
            } catch (err) {
                this.error = err.message;
                console.error('Failed to load drug groups', err);
            } finally {
                this.loading = false;
            }
        },
    },
});

```

```

    })
    frontend/src/modules/medicalHistory/components/CollapsibleSection/CollapsibleSection.js
export default {
  name: 'CollapsibleSection',
  props: {
    title: { type: String, required: true },
    preview: { type: String, default: '' },
    expanded: { type: Boolean, default: false },
    forceExpand: { type: Boolean, default: false },
  },
  data() {
    return {
      isExpanded: this.expanded,
    };
  },
  watch: {
    forceExpand(val) {
      if (val) {
        this.isExpanded = true;
      }
    },
  },
  methods: {
    toggle() {
      this.isExpanded = !this.isExpanded;
    },
    expand() {
      this.isExpanded = true;
    },
    collapse() {
      this.isExpanded = false;
    },
  },
};

frontend/src/modules/shared/api/drug.js
import apiClient from './client';

export const drugApi = {
  getAll(params = {}) {
    return apiClient.get('/drugs', { params }).then(res => res.data);
  },
};

frontend/src/modules/medicalHistory/components/AppointmentAdd/AppointmentAdd.js
<template src="./AppointmentAdd.template.html"></template>
<script src="./AppointmentAdd.script.js"></script>
<style scoped src="./AppointmentAdd.style.css"></style>
frontend/src/modules/medicalHistory/components/CollapsibleSection/CollapsibleSection.js
<template src="./CollapsibleSection.template.html"></template>

```

```
<script src="./CollapsibleSection.script.js"></script>  
<style scoped src="./CollapsibleSection.style.css"></style>
```